

MultiAgent Data Quality System



Eliya Allam, Simone Filosofi, Mattia Cervelli



Content Overview

- The Challenge
- Our Multi-Agent Solution
- NoIPA Scraping
- The full architecture behind
- **Profiler** Agent
- **Semantic** Agent
- **NaN Handler** Agent
- **Duplicate Columns** Agent
- **Consistency** Agent
- Unified **Remediation** Agent
- **Anomaly** Agent



The Challenge

NoiPA processes payroll and tax data for every employee of the Italian Public Administration.

It continuously ingests **heterogeneous datasets** (CSV files, JSON feeds, raw databases) spanning demographic, economic and health information.

Yet validation today is manual or nonexistent. Schema mismatches, missing values, format inconsistencies, duplicates and statistical outliers flow undetected into the systems that compute salaries and tax obligations for millions of public sector employees.

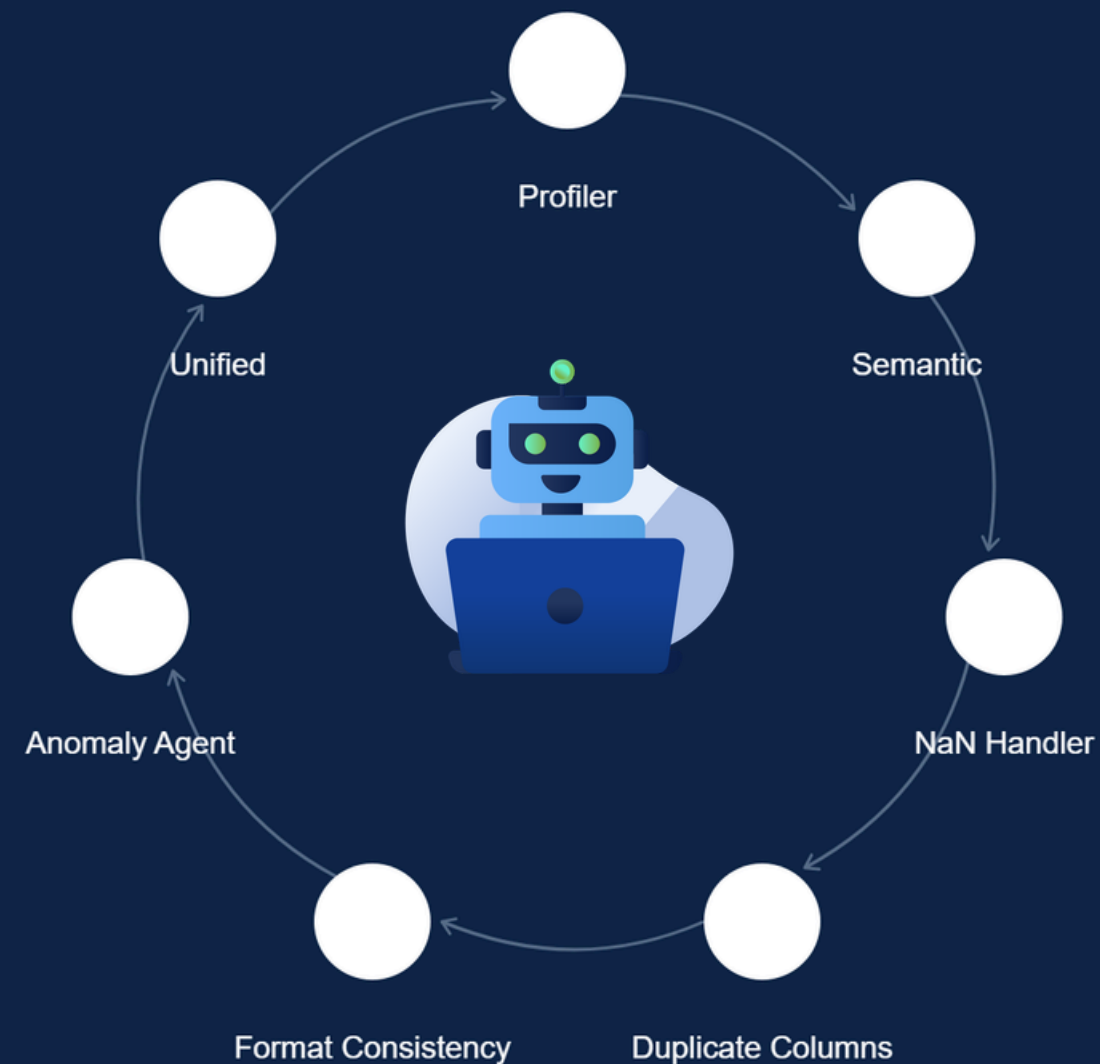




Our Multi-Agent Solution

Overview of collaborative agents for data quality

Seven specialized agents work together in a **synergistic approach** to ensure high data quality through comprehensive validation, analysis, and remediation processes tailored for NoiPA's needs.





59
Datasets

4
Domains

54
Unique Columns

1000+
enum values



NoIPA Scrapping (semi-RAG)

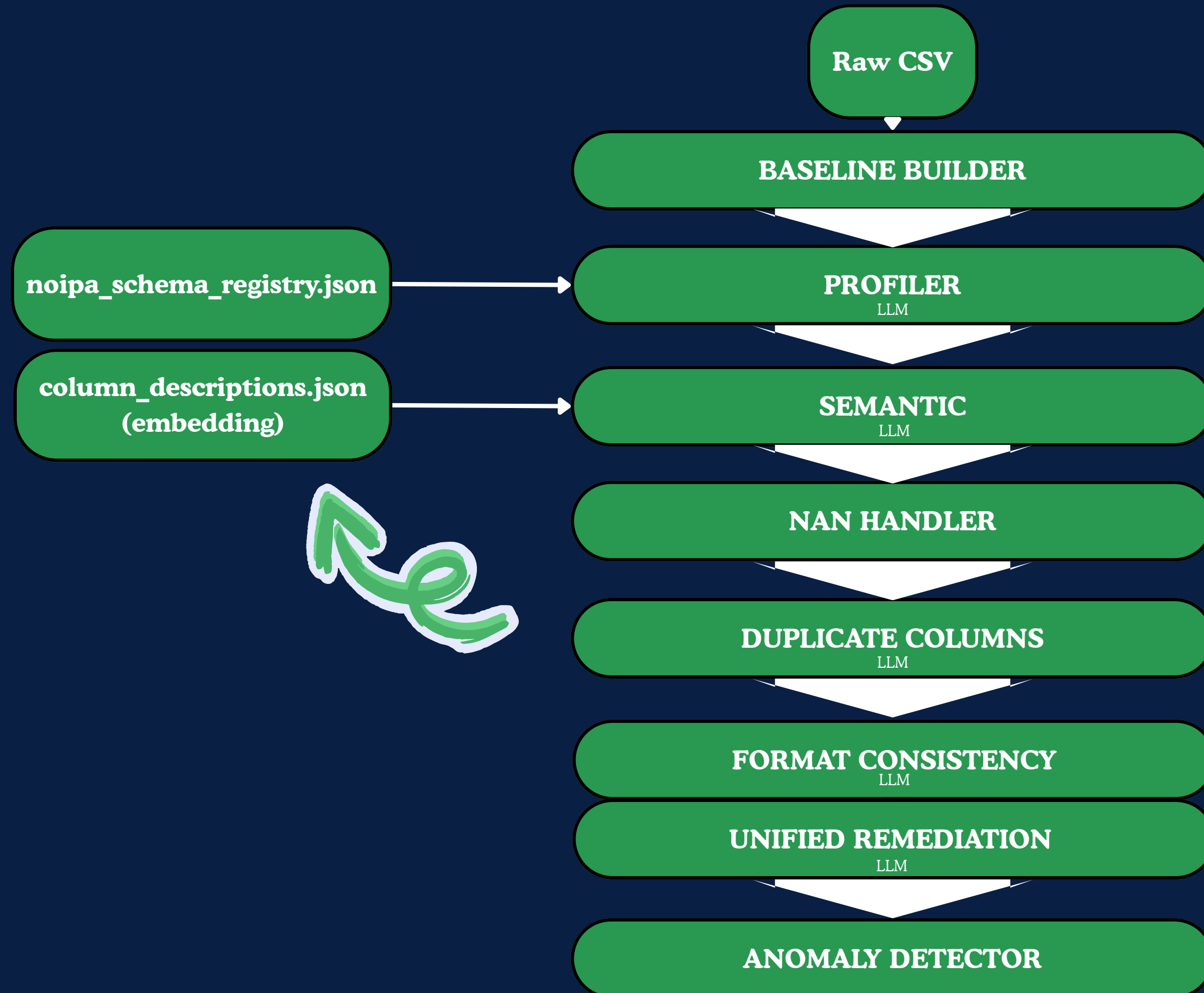
*We didn't ask NoiPA for the schema.
We derived it.*

Real World Quirks Formalised	
K-Anonymity Floor	Privacy Threshold of 6 on person counts
Ministry-Naming Drift	Pre/Post 2022 minsitry rename handled
Editorial Conventions	Italian lowercase conjunctions preserved
Schema-twin patterns	Identical Schemas, different semantics



Three Layers of Knowledge	
noipa_schema_registry.json	Validation Rules - dtype - format - nullability
baseline.json	Per-dataset blueprints with shared definitions
column_descriptions.json	Human-readable semantic context

Cascade inference - deterministic first, LLM last	
Tier 1 - Registry Lookup	Fast, auditable, dominant path
Tier 2 - RAG Over Descriptions	Semantic match for unknown columns
Tier 3 - LLM Inference	Rare, never breaks





Profiler Agent

Dataset ingestion

Loads raw CSV and resolves baseline.json schema into memory

Language detection

LLM detects Italian vs. English column naming

Domain classifier

The LLM matches the input columns against their signatures and assigns the correct domain, gating all downstream validation logic.

noipa_schema_registry.json 4 NoiPA domains:

Amministrati

Amministrazioni

Rapporti di Lavoro

Trattamento Economico





Semantic Agent

Columns description

All columns are sent in one batch to the LLM, which returns a **one-sentence** description per column. This description captures meaning and becomes the query for retrieval.

Embedding retrieval

Each column is embedded and scored against the 54 canonical NoiPA entries in *column_descriptions.json* by **cosine similarity**, **sample overlap** and **dtype match**. The top-5 candidates are returned.

LLM verdict

The canonical match is verified against the catalog; if invalid, it's discarded. The **registry dtype** overrides the LLM's suggestion and the column is cast in-place

Validation & casting

The LLM receives the column, its samples and the top-5 candidates, then picks the **best canonical match**. It returns dtype, meaning, placeholders and target casing.

The LLM response is enforced as a typed **Pydantic** model (*_SemanticResponse*), guaranteeing that *canonical_match*, *dtype*, *placeholders* and *target_casing* always come back in the exact shape the pipeline expects (no parsing, no surprises).



NaN Handler Agent

Placeholder replacement

For each column, the agent takes the **placeholder list** produced by the Semantic agent and replaces every **matching** value with pd.NA. It covers both **exact and case-insensitive** matches.

Nullability check

For columns where the registry specifies **is_nullable = false**, the agent counts any remaining nulls after replacement. Each violation is recorded as a ValidationReport and passed to downstream agents.

No LLM call is made at this step: all the intelligence was already applied by the Semantic agent upstream.



Duplicate Columns Agent

Similarity detection



All column pairs are scored by value-overlap **Jaccard similarity**. Any pair exceeding **0.80** is grouped together as a near-duplicate cluster.

Data survivor election

Within each group, the column with the **fewest nulls** is elected as the data survivor. Missing values in the survivor are **backfilled** from the other group members before the rest are dropped.

LLM name election

The LLM receives the duplicate group and the list of canonical column names from the NoiPA registry for the detected domain. It picks the **name most consistent with the registry** and returns a rationale. The survivor column is then renamed accordingly.



Format Consistency Agent

Format spec inference

For each column, the agent profiles the actual data distribution to derive a **format spec** (enum, regex or range), then refines it with an LLM call that also receives the **baseline hint from the registry**. The *stricter of the two specs wins*.

Functional dependency mining

For columns that still have nulls, the agent **deterministically** mines functional dependencies against their related columns (identified by the Semantic agent). If a lookup mapping exists, it surfaces an **ImputationHint** so the Unified agent can fill the gaps without guessing.

Violation detection

Every value in the column is validated against the resolved spec. Each offending value is recorded as a **FormatViolation** with its row index and observed value. Residual nulls not already flagged by the NaN Handler are also recorded here.

LLM corrections

The unique offending values are sent to the LLM alongside a sample of valid values and the expected pattern. The **LLM proposes** a **targeted correction** for each bad value: these are stored as `value_corrections` for the downstream Unified agent to apply.



Unified Remediation Agent



Grouping

Columns are grouped by transitive closure of their *related_columns* links. Each group is treated as one unit and fixes are proposed at group level so cross-column dependencies are handled together.

Fix proposal

The LLM receives each group's violations, evidence rows, clean reference rows, value corrections from Format Consistency and imputation hints. It returns a **structured set of fix proposals**, each scoped to **specific columns** and linked to the violations it addresses.

Human approval gate

The proposed fixes are surfaced in the Streamlit UI for **human review**. For each fix the user can: **Accept** it to queue it for execution, **Reject** it to discard it, or **Edit** it by typing *feedback that triggers a fresh LLM re-proposal*. Only accepted fixes are applied to the DataFrame.

Sandbox trial + self-review

Each proposal is dry-run in an **isolated sandbox (E2B)**. The same model then reviews the trial outcome — rows changed, violations resolved — and either approves or rejects with feedback (**max. 2 iterations**).
On rejection, the fix is regenerated once using the reviewer's notes, then finalized regardless.





Anomaly Agent

Numeric outlier detection

For every numeric column, the agent computes Q1, Q3 and **IQR**. Any value outside the bounds is flagged as an outlier. Up to 50 entries per column are stored with their row index and the exact bounds that were violated.

Rare category detection

For every string or categorical column, the agent computes **value frequencies**. Any value appearing fewer than 3 times or below 1% of non-null rows is flagged as a rare category. The stats block records total distinct values, rare count and the top-2 most frequent values for context.

LLM comment

Once all columns are processed, the agent sends the aggregated findings to LLM in a single batch call. The model returns one **explanatory sentence** per flagged column, contextualising whether the anomaly is likely noise, a data entry error, or a legitimate edge case.

No changes are made to the DataFrame at this step: this is a pure detection step.
Results are written to `anomaly_report.json` for the report generator to consume.



Where We Excel

1

Empirically Derived:

*Knowledge-base grows with NoIPA
Backed by real-data
Incredible Foundation*

Schema discovered from real data –
not idealized specs.

2

Deterministic-First

*3 tiers
Human-in-the-loop auditable
Cost-efficient at scale*

LLM where really adds value, not as first
response.

3

Repair-First:

*Multi-Signal canonical matching
Backfill survivor before dropping*

We don't just flag bad data, we fix it
safely

4

Italian PA-native:

*Lowercase conjunctions preserved
HR/Payroll aware anomalies*

Built around the quirks of
Italian public administration data





What we would implement next

Extending the schema registry

Adding **more datasets** to the registry, leading to more shared columns definitions that would **improve canonical matching** accuracy.

Human-in-the-loop on semantic matching

When Semantic Agent returns a low canonical match, allows a **domain expert** to confirm or override it before rest of pipeline runs.
(with a larger complete schema registry maybe this wouldn't be needed)

Full NaN imputation

Instead of only filling nulls when a related column provides a deterministic lookup, **LLM-guided interpolation** would cover cases without dependency. Use of smarter LLMs is needed (we used just gpt 4o-mini)

Learnable Anomaly Thresholds

Domain-specific thresholds learned from labelled historical NoIPA data would **reduce false-positives** significantly.

Active Learning Loop

When the LLM resolves an unfamiliar column, the resolution get reviewed and auto-promoted into schema_registry.json.



Our demo app

The good news: everything is committed and timestamped before the deadline.

The bad news: our correct app is on the wrong branch.

The very good news: you now know we can use version control.



```
git checkout 06493d1
```



Contact Us

Email

eliya.allam@studenti.luiss.it
simone.filosofi@studenti.luiss.it
mattia.cervelli@studenti.luiss.it

Thank you for you
time